FORMATIVE ASSESSMENT 1

Research assignment

Programming with C#

PRG521

2024

Chane Marais

20231494

CTU training solutions

Creative & Technology Universitas

Table of Contents

Programming with C# ..... 1 PRG521 ..... 1 .....

Section A: Multiple Choice Questions (40 Marks)..... 4

End of section A. [Total = 40 Marks] 8

Section B: Scenario Questions (60 Marks) 8

End of section B. [Total = 60 Marks] 9

End of paper..... 9

Please complete the declaration of authenticity below for all assignments:

DECLARATION OF AUTHENTICITY

I Chane Marais hereby

declare that the contents of this assignment are entirely my own work with the exception of the following elements: (List the elements of work in this project that were not self-generated as well as who the originator of the element is)

Element	Originator

Signature: CM Date: 11June

Software Development

Module: Programming with C#

Module Code: PRG521

NQF: 5

Credits: 20

Assessment type: Formative Assessment

Mark allocation: 100

Hand out date: 15 March 2024, 08:00

Hand in date: 01 April 2024, 23:30

Instructions:

Read each question carefully and consider the mark allocation prior to answering.

- Ensure you answer all questions.
- Section A: Multiple choice questions – **40** marks.
- Section B: Short Questions – **60** marks.
- For final submission**,** include the Declaration of Authenticity **and your Answer Sheet (PDF Document)**

Unit Standards:

- Create a simple C# program
- Understand Variables
- Understand the Visual C# programming and user environment
- Describe the Data Types in C#
- Differentiate between numeric and fundamental types
- Understanding conversions between data types
- Understanding categories of types
- Declare types that allow null
- Using typed local variables
- Create and use tuples
- Create and use arrays
- Understanding flow control
- Create control flow statements
- Using Jump statements
- Understandings C# pre-processor directives
- Describe code blocks, scopes and declaration spaces

Section A: Multiple Choice Questions (40 Marks)

Instructions: Choose the correct option for each question below.

1. Which of the following statements correctly performs an explicit conversion of a double to an int in C#? (2 Marks)

   A. `int i = (int)double d;` B. `int i = int.Parse(double d);` C. `int i = Convert.ToInt32(double d);`
   D. `int i = double d as int;`

2. What will happen if you try to implicitly convert a long to a short when the long value is out of the shortrange? (2 Marks)

A. A compile-time error
B. A runtime exception C. Data loss due to truncation
D. Automatic rounding to the nearest valid short value

3. Which of the following is true about implicit conversion in C#? (2 Marks)

A. It requires special syntax to be performed.
B. It is always safe and never loses information.
C. It can only be done between numeric data types.
D. It can lead to compile-time errors if there is a risk of data loss.

4. What is the correct way to convert a string to a bool in C#? (2 Marks)

A. `bool b = bool.Parse("true");`
B. `bool b = (bool)"true";`
C. `bool b = Convert.ToBoolean("true");`
D. Both A and C are correct.

5. Which of the following is the correct way to explicitly convert a double to an int in C#? (2 Marks)

A. `int i = convert.ToInt(doubleVar);`
B. `int i = (int)doubleVar;` C. `int i = int.Parse(doubleVar);`
D. `int i = int.convert(doubleVar);`

6. What will happen if you try to implicitly convert a long to an int when the value of the long variable is outside the range of an int? (2 Marks)

A. A compile-time error
B. A runtime error
C. The value is converted without any error
D. Data loss without any error

7. Which method is used for converting a string representation of a number to its 32-bit signed integer equivalent? (2 Marks)

A. `Int32.Parse()` B. `Convert.ToInt32()`
C. `Int32.TryParse()`
D. All of the above

8. In C#, which of the following is NOT a valid explicit conversion? (2 Marks)

A. From double to float
B. From char to int C. From bool to int
D. From float to double

9. Which preprocessor directive is used to define a symbol in C#? (2 Marks)

A. #define
B. #if C. #else
D. #endif

10. What is the purpose of the #region and #endregion directives in C#? (2 Marks)

A. To create a block of code that can be expanded or collapsed when using supporting IDEs
B. To include a specific namespace automatically in all files
C. To conditionally compile a block of code
D. To define symbols for conditional compilation

11. Which of the following directives is used to give a warning message during compilation? (2 Marks)

A. #error
B. #warning
C. #pragma
D. #line

12. The #if directive in C# is used for: (2 Marks)

A. Defining symbols.
B. Conditional compilation.
C. Creating namespaces.
D. Error handling.

13. Which of the following statements about declaration spaces in C# is true? (2 Marks)

A. A namespace can contain two classes with the same name.
B. A class can contain two methods with the same name and parameter types.
C. A method can have two local variables with the same name within the same scope.
D. A block of code can define a local variable with the same name as one of its method parameters.

14. What is the scope of a public class member variable in C#? (2 Marks)

A. Only within the class it is declared
B. Within the class and derived classes
C. Anywhere in the application
D. Within the same assembly

15. A local variable declared inside a loop: (2 Marks)

A. Is accessible only within the loop
B. Can be accessed anywhere in the method containing the loop
C. Is accessible in the entire class
D. Can be accessed in any method within the same namespace

16. In C#, if a variable is declared within an if statement block, then: (2 Marks)

A. It can be accessed anywhere in the block containing the if statement
B. Its scope ends when the if statement block ends
C. It can be accessed by all methods in the class
D. Its scope is throughout the method that contains the if statement

17. Which statement is used in C# to jump to the next iteration of the closest enclosing loop? (2 Marks)

A. `break;`
B. `continue;`
C. `goto;`
D. `return;`

18. What does the break statement do in a switch statement in C#? (2 Marks)

A. Pauses the execution of the case
B. Exits the current method
C. Terminates the switch statement
D. Skips to the next case

19. Which of the following is NOT a control flow statement in C#? (2 Marks)

A. `if-else`
B. `try-catch`

C. `for`
D. `#region`

20. The goto statement in C#: (2 Marks)

A. Is used to break out of a loop
B. Can transfer control to a specified label within a method
C. Is used to continue to the next iteration of a loop
D. Can only be used inside switch statements

End of section A. **[Total = 40 Marks]**

Section B: Scenario Questions (60 Marks)

Instructions: Complete the following task and attach your code snippet or screenshots to your answer sheet.

UXI Mortgage Loans

Imagine you're working as a software developer for a fintech company that specializes in providing financial planning tools. The company is currently working on a project to develop a comprehensive mortgage calculator application that will help users estimate their monthly mortgage payments, including the principal and interest over the life of the loan. Your task is to build the core functionality of this mortgage calculator.

The mortgage calculator should take into account the total loan amount, the interest rate (annual), the term of the loan in years, and optionally, the start date of the loan. The calculator should provide the following outputs:

- Monthly repayment amount.
- Total amount of interest paid over the life of the loan.
- Total amount paid over the life of the loan (principal + interest).
- An amortization schedule that shows the breakdown of principal and interest for each month until the loan is paid off.
- The company wants this tool to be accurate and user-friendly, providing valuable insights for users planning their finances.

Mashile T., (2024)

2.1 Your task is to develop a C# program that implements the mortgage calculator as described. You must create a class named `MortgageCalculator` that contains at least the following methods:

A. `CalculateMonthlyRepayment(double loanAmount, double annualInterestRate, int loanTermYears)`: This method should calculate and return the monthly repayment amount.
B. `CalculateTotalInterestPaid(double loanAmount, double annualInterestRate, int loanTermYears)`: This method should calculate and return the total amount of interest paid over the life of the loan.
C. `CalculateTotalAmountPaid(double loanAmount, double annualInterestRate, int loanTermYears)`: This method should calculate and return the total amount paid over the life of the loan.
D. `GenerateAmortizationSchedule(double loanAmount, double annualInterestRate, int loanTermYears)`: This method should generate and return an amortization schedule. Each entry in the schedule should detail the payment number, payment amount, interest paid, principal paid, and remaining balance.
E. Consider using appropriate data structures for storing and returning the amortization schedule. You may also add any additional methods or properties you think are necessary to make the `MortgageCalculator` class functional and user-friendly.

Notes:

- Use appropriate data types for all calculations and return values.
- Ensure the program handles different loan terms (e.g., 15 years, 30 years) and interest rates accurately.
- The interest rate input should be considered as an annual rate.
- The amortization schedule should be detailed and easy to understand for the end-user.
- Include comments in your code to explain the logic behind your calculations and any assumptions made.

End of section B. **[Total = 60 Marks]**

End of paper.

Question 2 Rubric

Evaluation Criterion	Mark Allocation	Mark Awarded
Correctness		
The program accurately calculates the: monthly repayment amount, total interest paid, total amount paid, and generates a correct amortization schedule as per the provided specifications.	10	
Functionality		
The program correctly handles edge cases, such as zero interest rates, a loan term of zero years, or extremely high loan amounts.	10	

The code efficiently performs calculations without unnecessary computations or memory usage.	10	
The `MortgageCalculator` class is designed logically with methods and properties that reflect the task requirements. Proper use of access modifiers (public, private) is evaluated.	10	
The amortization schedule is presented in a clear, understandable format for the user, with appropriate labels for each column.	10	
Usability		
The program includes a simple and intuitive way for users to input data and view results. For console applications, this includes clear prompts and instructions.	10	
TOTAL		**/60**

Answers

Section A: Multiple choice questions – **40** marks.

1. A
2. A
3. D
4. A
5. B
6. D
7. B
8. D
9. A
10. A
11. B
12. B
13. D
14. C
15. A
16. B
17. B
18. C
19. D
20. B

Section B: Short Questions – **60** marks.

Program › No selection Source

```
using System;
using System.Collections.Generic;

namespace MortgageCalculatorApp
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Create an instance of the MortgageCalculator
                MortgageCalculator calculator = new MortgageCalculator();

                // Read inputs from the user
                Console.WriteLine("Enter the total loan amount:");
                double loanAmount = double.Parse(Console.ReadLine());

                Console.WriteLine("Enter the annual interest rate (in %):");
                double annualInterestRate = double.Parse(Console.ReadLine());

                Console.WriteLine("Enter the term of the loan in years:");
                int loanTermYears = int.Parse(Console.ReadLine());

                // Calculate and display monthly repayment
                double monthlyRepayment = calculator.CalculateMonthlyRepayment(loanAmount, annualInterestRate, loanTermYears);
                Console.WriteLine($"Monthly Repayment Amount: {monthlyRepayment:C}");

                // Calculate and display total interest paid
                double totalInterestPaid = calculator.CalculateTotalInterestPaid(loanAmount, annualInterestRate, loanTermYears);
                Console.WriteLine($"Total Interest Paid: {totalInterestPaid:C}");

                // Calculate and display total amount paid
```

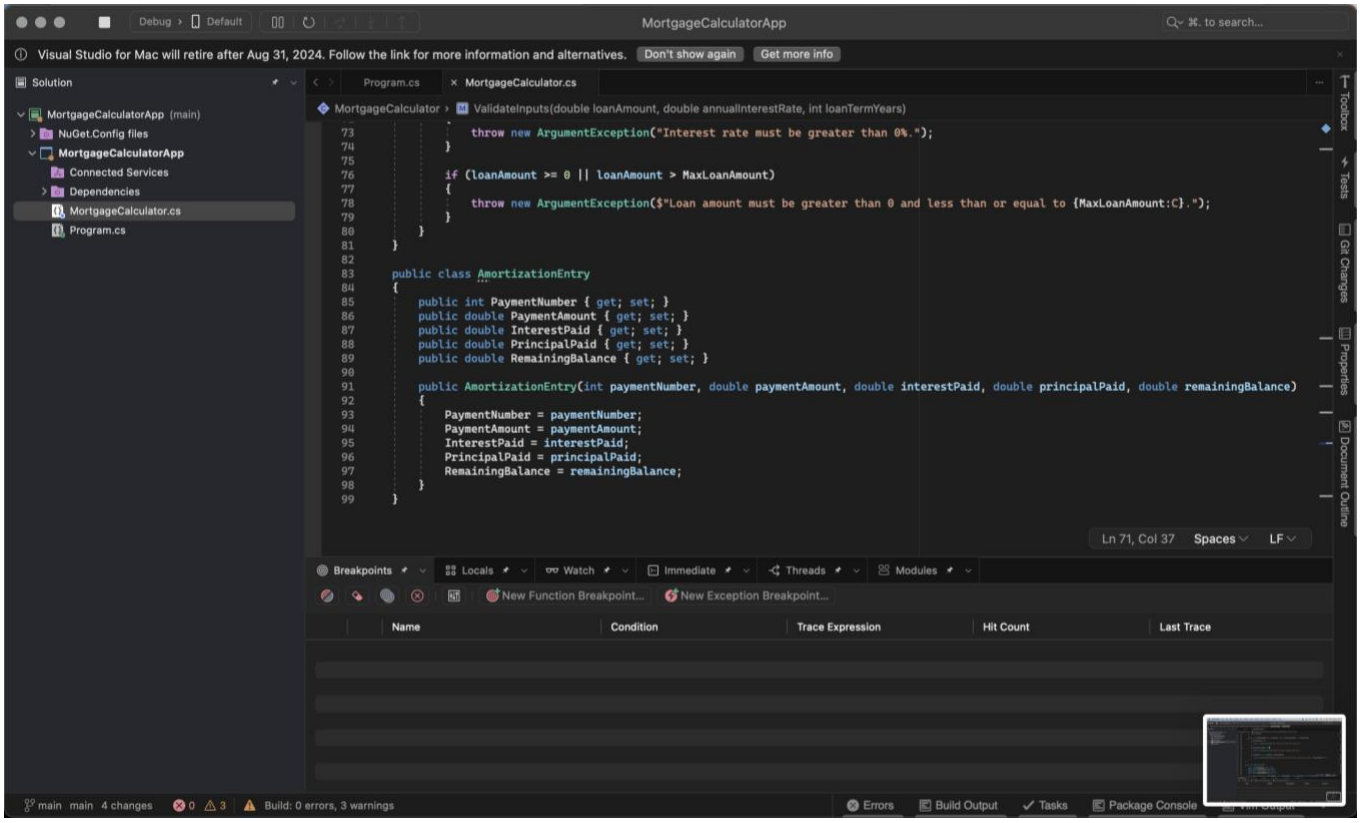
Ln 6, Col 18 Spaces LF

```
            double totalInterestPaid = calculator.CalculateTotalInterestPaid(loanAmount, annualInterestRate, loanTermYears);
            Console.WriteLine($"Total Interest Paid: {totalInterestPaid:C}");

            // Calculate and display total amount paid
            double totalAmountPaid = calculator.CalculateTotalAmountPaid(loanAmount, annualInterestRate, loanTermYears);
            Console.WriteLine($"Total Amount Paid: {totalAmountPaid:C}");

            // Generate and display amortization schedule
            List<AmortizationEntry> schedule = calculator.GenerateAmortizationSchedule(loanAmount, annualInterestRate, loanTermYears);
            Console.WriteLine("Amortization Schedule:");
            Console.WriteLine("Payment#\tPayment Amount\tInterest Paid\tPrincipal Paid\tRemaining Balance");
            foreach (var entry in schedule)
            {
                Console.WriteLine($"{entry.PaymentNumber}\t\t{entry.PaymentAmount:C}\t\t{entry.InterestPaid:C}\t\t{entry.PrincipalPaid:C
            }
        }
        catch (FormatException)
        {
            Console.WriteLine("Please enter valid numeric values.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"An error occurred: {ex.Message}");
        }

        // Wait for the user to press Enter before closing the console window
        Console.WriteLine("Press Enter to exit.");
        Console.ReadLine();
    }
  }
}
```

```
using System;
using System.Collections.Generic;

public class MortgageCalculator
{
    private const int MonthsInYear = 12;
    private const double MaxLoanAmount

    public double CalculateMonthlyRepay                    e annualInterestRate, int loanTermYears)
    {
        ValidateInputs(loanAmount, annualInterestRate, loanTermYears);

        double monthlyInterestRate = annualInterestRate / 100 / MonthsInYear;
        int numberOfPayments = loanTermYears * MonthsInYear;

        double monthlyRepayment = loanAmount * (monthlyInterestRate * Math.Pow(1 + monthlyInterestRate, numberOfPayments))
                                  / (Math.Pow(1 + monthlyInterestRate, numberOfPayments) - 1);

        return monthlyRepayment;
    }

    public double CalculateTotalInterestPaid(double loanAmount, double annualInterestRate, int loanTermYears)
    {
        ValidateInputs(loanAmount, annualInterestRate, loanTermYears);

        double monthlyRepayment = CalculateMonthlyRepayment(loanAmount, annualInterestRate, loanTermYears);
        int numberOfPayments = loanTermYears * MonthsInYear;
        double totalRepayment = monthlyRepayment * numberOfPayments;
        double totalInterestPaid = totalRepayment - loanAmount;
        return totalInterestPaid;
    }
```

```
32
33          public double CalculateTotalAmountPaid(double loanAmount, double annualInterestRate, int loanTermYears)
34          {
35              ValidateInputs(loanAmount, annualInterestRate, loanTermYears);
36
37              double monthlyRepayment = CalculateMonthlyRepayment(loanAmount, annualInterestRate, loanTermYears);
38              int numberOfPayments = loanTermYears * MonthsInYear;
39              double totalAmountPaid = monthlyRepayment * numberOfPayments;
40              return totalAmountPaid;
41          }
42
43          public List<AmortizationEntry> GenerateAmortizationSchedule(double loanAmount, double annualInterestRate, int loanTermYears)
44          {
45              ValidateInputs(loanAmount, annualInterestRate, loanTermYears);
46
47              List<AmortizationEntry> schedule = new List<AmortizationEntry>();
48              double monthlyInterestRate = annualInterestRate / 100 / MonthsInYear;
49              int numberOfPayments = loanTermYears * MonthsInYear;
50              double monthlyRepayment = CalculateMonthlyRepayment(loanAmount, annualInterestRate, loanTermYears);
51              double remainingBalance = loanAmount;
52
53              for (int month = 1; month <= numberOfPayments; month++)
54              {
55                  double interestPaid = remainingBalance * monthlyInterestRate;
56                  double principalPaid = monthlyRepayment - interestPaid;
57                  remainingBalance -= principalPaid;
58                  AmortizationEntry entry = new AmortizationEntry(month, monthlyRepayment, interestPaid, principalPaid, remainingBalance);
59                  schedule.Add(entry);
60              }
61              return schedule;
62          }
```

```
61              return schedule;
62          }
63
64          private void ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
65          {
66              if (loanTermYears <= 0)
67              {
68                  throw new ArgumentException("Loan term must be greater than 0 years.");
69              }
70
71              if (annualInterestRate <= 0)
72              {
73                  throw new ArgumentException("Interest rate must be greater than 0%.");
74              }
75
76              if (loanAmount >= 0 || loanAmount > MaxLoanAmount)
77              {
78                  throw new ArgumentException($"Loan amount must be greater than 0 and less than or equal to {MaxLoanAmount:C}.");
79              }
80          }
81      }
82
83      public class AmortizationEntry
84      {
85          public int PaymentNumber { get; set; }
86          public double PaymentAmount { get; set; }
87          public double InterestPaid { get; set; }
88          public double PrincipalPaid { get; set; }
89          public double RemainingBalance { get; set; }
90
91          public AmortizationEntry(int paymentNumber, double paymentAmount, double interestPaid, double principalPaid, double remainingBalance)
```


MortgageCalculatorApp
Visual Studio for Mac will retire after Aug 31, 2024. Follow the link for more information and alternatives.
Don't show again
Get more info
Solution
MortgageCalculatorApp (main)
NuGet.Config files
MortgageCalculatorApp
Connected Services
Dependencies
MortgageCalculator.cs
Program.cs
MortgageCalculator › ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
throw new ArgumentException("Interest rate must be greater than 0%.");
if (loanAmount >= 0 || loanAmount > MaxLoanAmount)
throw new ArgumentException($"Loan amount must be greater than 0 and less than or equal to {MaxLoanAmount:C}.");
public class AmortizationEntry
public int PaymentNumber { get; set; }
public double PaymentAmount { get; set; }
public double InterestPaid { get; set; }
public double PrincipalPaid { get; set; }
public double RemainingBalance { get; set; }
public AmortizationEntry(int paymentNumber, double paymentAmount, double interestPaid, double principalPaid, double remainingBalance)
PaymentNumber = paymentNumber;
PaymentAmount = paymentAmount;
InterestPaid = interestPaid;
PprincipalPaid = principalPaid;
RemainingBalance = remainingBalance;
Ln 71, Col 37
Spaces
LF
Breakpoints
Locals
Watch
Immediate
Threads
Modules
New Function Breakpoint...
New Exception Breakpoint...
Name
Condition
Trace Expression
Hit Count
Last Trace
main main 4 changes
Build: 0 errors, 3 warnings
Errors
Build Output
Tasks
Package Console

Visual Studio for Mac will retire after Aug 31, 2024. Follow the link for more information and alternatives. Don't show again Get more info

Solution

- MortgageCalculatorApp (main)
  - NuGet.Config files
  - MortgageCalculatorApp
    - Connected Services
    - Dependencies
    - MortgageCalculator.cs
    - Program.cs

Program.cs × MortgageCalculator.cs

MortgageCalculator › ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)

```
53          for (int month = 1; month <= numberOfPayments; month++)
54          {
55              double interestPaid = remainingBalance * monthlyInterestRate;
56              double principalPaid = monthlyRepayment - interestPaid;
57              remainingBalance -= principalPaid;
58              AmortizationEntry entry = new AmortizationEntry(month, monthlyRepayment, interestPaid, principalPaid, remainingBalance);
59              schedule.Add(entry);
60          }
61          return schedule;
62      }
63
64      private void ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
65      {
```

Terminal – MortgageCalculatorApp

```
Enter the total loan amount:
100
Enter the annual interest rate (in %):
10
Enter the term of the loan in years:
10
Monthly Repayment Amount: $1.32
Total Interest Paid: $58.58
Total Amount Paid: $158.58
Amortization Schedule:
Payment#        Payment Amount  Interest Paid   Principal Paid  Remaining Balance
1               $1.32           $0.83           $0.49           $99.51
2               $1.32           $0.83           $0.49           $99.02
3               $1.32           $0.83           $0.50           $98.52
4               $1.32           $0.82           $0.50           $98.02
5               $1.32           $0.82           $0.50           $97.52
6               $1.32           $0.81           $0.51           $97.01
7               $1.32           $0.81           $0.51           $96.50
8               $1.32           $0.80           $0.52           $95.98
9               $1.32           $0.80           $0.52           $95.46
10              $1.32           $0.80           $0.53           $94.93
11              $1.32           $0.79           $0.53
12              $1.32           $0.79           $0.53
13              $1.32           $0.78           $0.54           $93.33
14              $1.32           $0.78           $0.54           $92.78
15              $1.32           $0.77           $0.55           $92.23
16              $1.32           $0.77           $0.55           $91.68
17              $1.32           $0.76           $0.56           $91.12
18              $1.32           $0.76           $0.56           $90.56
19              $1.32           $0.75           $0.57           $90.00
20              $1.32           $0.75           $0.57           $89.42
21              $1.32           $0.75           $0.58           $88.85
22              $1.32           $0.74           $0.58           $88.27
23              $1.32           $0.74           $0.59           $87.68
24              $1.32           $0.73           $0.59           $87.09
```

Enable or Disable All Breakpoints

main main 4 changes 0 3 Build: 0 errors, 3 warnings Errors Build Output Tasks Package Console Vim Output

Debug › Default MortgageCalculatorApp ⌘. to search...

Visual Studio for Mac will retire after Aug 31, 2024. Follow the link for more information and alternatives. Don't show again Get more info

Solution

- MortgageCalculatorApp (main)
  - NuGet.Config files
  - MortgageCalculatorApp
    - Connected Services
    - Dependencies
    - MortgageCalculator.cs
    - Program.cs

Program.cs × MortgageCalculator.cs

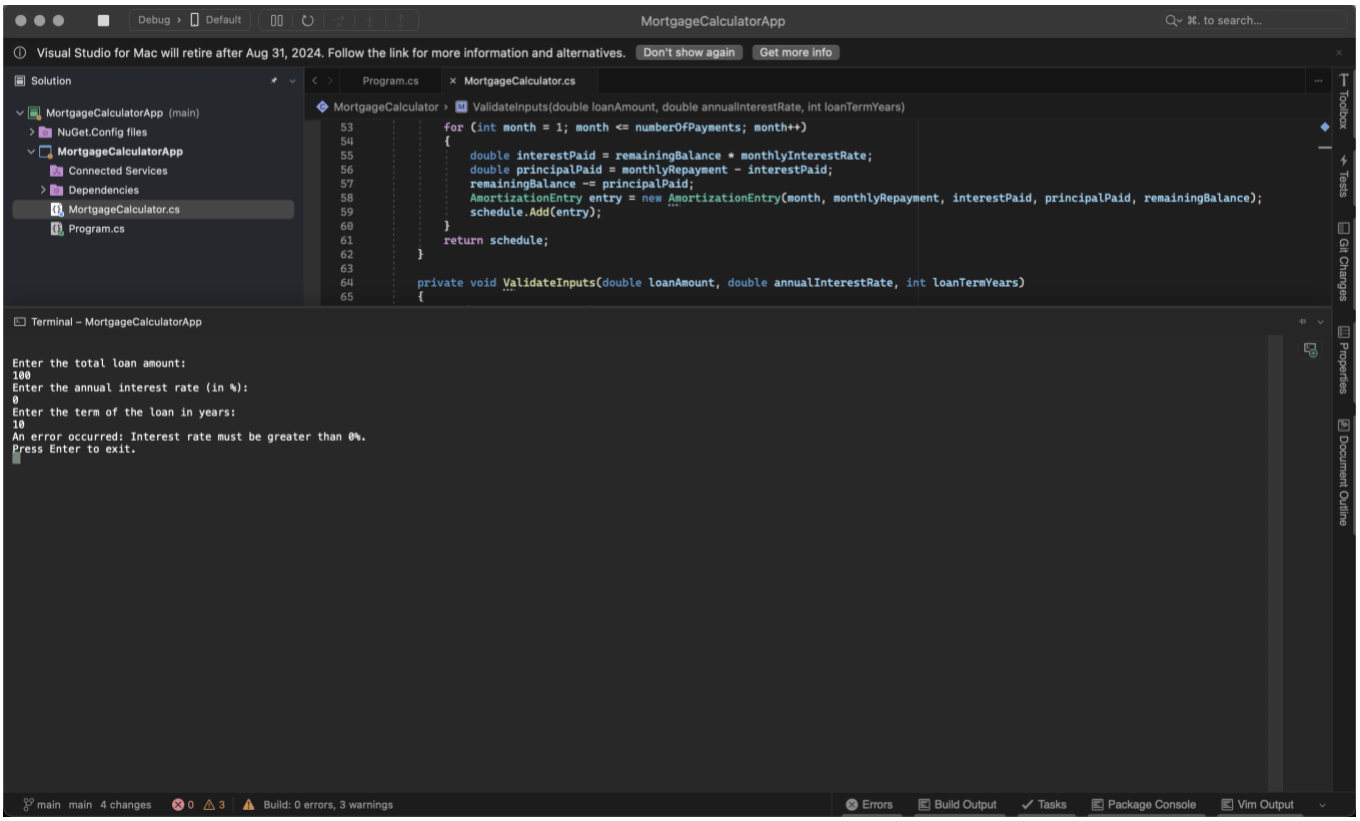
MortgageCalculator › ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)

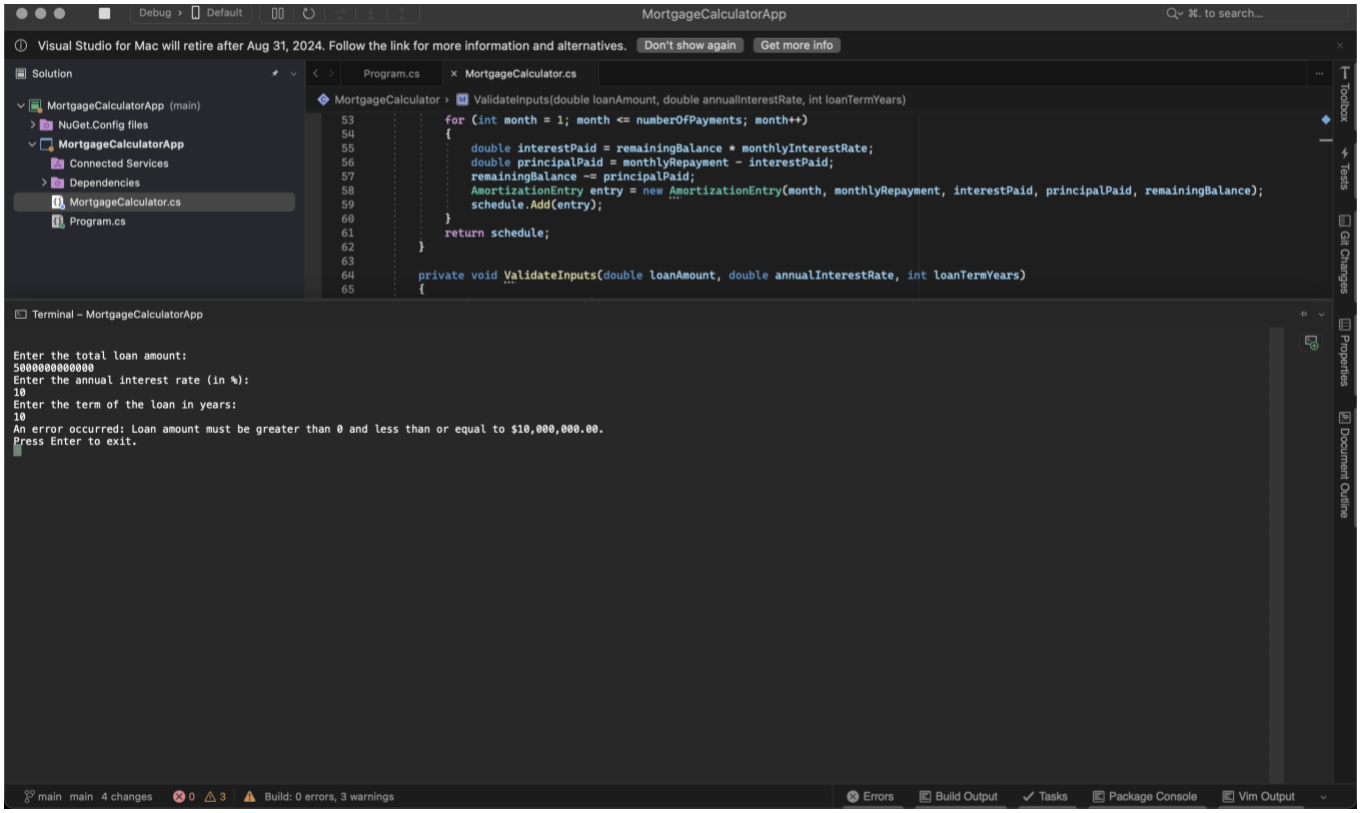
```
53          for (int month = 1; month <= numberOfPayments; month++)
54          {
55              double interestPaid = remainingBalance * monthlyInterestRate;
56              double principalPaid = monthlyRepayment - interestPaid;
57              remainingBalance -= principalPaid;
58              AmortizationEntry entry = new AmortizationEntry(month, monthlyRepayment, interestPaid, principalPaid, remainingBalance);
59              schedule.Add(entry);
60          }
61          return schedule;
62      }
63
64      private void ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
65      {
```

Terminal – MortgageCalculatorApp

```
Enter the total loan amount:
100
Enter the annual interest rate (in %):
10
Enter the term of the loan in years:
0
An error occurred: Loan term must be greater than 0 years.
Press Enter to exit.
```

main main 4 changes 0 3 Build: 0 errors, 3 warnings Errors Build Output Tasks Package Console Vim Output


MortgageCalculatorApp
Visual Studio for Mac will retire after Aug 31, 2024. Follow the link for more information and alternatives.
Don't show again
Get more info
Solution
MortgageCalculatorApp (main)
NuGet.Config files
MortgageCalculatorApp
Connected Services
Dependencies
MortgageCalculator.cs
Program.cs
MortgageCalculator › ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
53 for (int month = 1; month <= numberOfPayments; month++)
54 {
55 double interestPaid = remainingBalance * monthlyInterestRate;
56 double principalPaid = monthlyRepayment - interestPaid;
57 remainingBalance -= principalPaid;
58 AmortizationEntry entry = new AmortizationEntry(month, monthlyRepayment, interestPaid, principalPaid, remainingBalance);
59 schedule.Add(entry);
60 }
61 return schedule;
62 }
63
64 private void ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
65 {
Terminal – MortgageCalculatorApp
Enter the total loan amount:
100
Enter the annual interest rate (in %):
0
Enter the term of the loan in years:
10
An error occurred: Interest rate must be greater than 0%.
Press Enter to exit.
main main 4 changes
Build: 0 errors, 3 warnings
Errors
Build Output
Tasks
Package Console
Vim Output


MortgageCalculatorApp
Visual Studio for Mac will retire after Aug 31, 2024. Follow the link for more information and alternatives.
Don't show again
Get more info
Solution
MortgageCalculatorApp (main)
NuGet.Config files
MortgageCalculatorApp
Connected Services
Dependencies
MortgageCalculator.cs
Program.cs
MortgageCalculator › ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
53 for (int month = 1; month <= numberOfPayments; month++)
54 {
55 double interestPaid = remainingBalance * monthlyInterestRate;
56 double principalPaid = monthlyRepayment - interestPaid;
57 remainingBalance -= principalPaid;
58 AmortizationEntry entry = new AmortizationEntry(month, monthlyRepayment, interestPaid, principalPaid, remainingBalance);
59 schedule.Add(entry);
60 }
61 return schedule;
62 }
63
64 private void ValidateInputs(double loanAmount, double annualInterestRate, int loanTermYears)
65 {
Terminal – MortgageCalculatorApp
Enter the total loan amount:
5000000000000
Enter the annual interest rate (in %):
10
Enter the term of the loan in years:
10
An error occurred: Loan amount must be greater than 0 and less than or equal to $10,000,000.00.
Press Enter to exit.
main main 4 changes
Build: 0 errors, 3 warnings
Errors
Build Output
Tasks
Package Console
Vim Output